

SIMATS ENGINEERING

ASSESSMENT TOOL II – DATA ANALYSIS PROJECT

Course: Data Handling and Visualization	Code: DSA0601	CO: CO3
Duration: 120 min	Total Marks: 100	Reg No : 192324246 Name: S. Sharon Surya

COURSE OUTCOME COVERED

CO3: Analyze time-series data, trends, associations, and uncertainty through appropriate visualization methods. (BL4)

Activity Tool : Data Analysis Project

Weightage: 20%

Code of Conduct:

I S. V. K. Sharon Surya(192324246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Sharon Surya

Criteria	Problem Understanding & Data Preparation 25%	Data Analysis & Interpretation 30%	Application of Analytical Techniques 20%	Analysis of Results 15%	Organization & Documentation 10%	Presentation & Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Data Analysis Project

Question 1 – Distribution Analysis Using ECDF and Q-Q Plot

Dataset: Employee_Salaries.csv

Assessment Task

Using the Employee_Salaries dataset, analyze the distribution of employee salaries.

1. Create an empirical cumulative distribution function (ECDF) for Salary.
2. Create a Q-Q plot to assess whether Salary is approximately normally distributed.
3. Compare the salary distributions of the four departments using suitable distribution plots.
4. Identify any possible skewness or unusual observations and explain what the plots reveal.

Dataset:

Employee_ID	Department	Salary
E001	IT	42000
E002	IT	45000
E003	IT	48000
E004	IT	51000
E005	IT	55000
E006	HR	38000
E007	HR	40000
E008	HR	42000
E009	HR	46000
E010	HR	50000
E011	Finance	47000
E012	Finance	49000
E013	Finance	52000
E014	Finance	58000
E015	Finance	65000
E016	Marketing	36000
E017	Marketing	39000

E018	Marketing	43000
E019	Marketing	47000
E020	Marketing	72000

R Code:

```
# =====
# Q1: EMPLOYEE SALARY DISTRIBUTION
# =====
```

```
library(ggplot2)
library(dplyr)
```

```
# -----
# 1. Create Dataset
# -----
```

```
employee <- data.frame(
  Employee_ID = c(
    "E001","E002","E003","E004","E005",
    "E006","E007","E008","E009","E010",
    "E011","E012","E013","E014","E015",
    "E016","E017","E018","E019","E020"
  ),
```

```
  Department = c(
    rep("IT",5),
    rep("HR",5),
    rep("Finance",5),
    rep("Marketing",5)
  ),
```

```
  Salary = c(
    42000,45000,48000,51000,55000,
    38000,40000,42000,46000,50000,
    47000,49000,52000,58000,65000,
    36000,39000,43000,47000,72000
  )
)
```

```
)
```

```
# View data  
print(employee)
```

```
# -----
```

```
# 2. ECDF Plot
```

```
# -----
```

```
ggplot(employee, aes(x = Salary)) +  
  stat_ecdf(geom = "step") +  
  labs(  
    title = "Empirical Cumulative Distribution Function of Employee Salaries",  
    x = "Salary",  
    y = "Cumulative Probability"  
  ) +  
  theme_minimal()
```

```
# -----
```

```
# 3. Q-Q Plot
```

```
# -----
```

```
ggplot(employee, aes(sample = Salary)) +  
  stat_qq() +  
  stat_qq_line() +  
  labs(  
    title = "Q-Q Plot of Employee Salaries",  
    x = "Theoretical Quantiles",  
    y = "Sample Quantiles"  
  ) +  
  theme_minimal()
```

```
# -----
```

```
# 4. Department-wise Boxplot
```

```
# -----
```

```
ggplot(employee, aes(x = Department, y = Salary, fill = Department)) +  
  geom_boxplot() +  
  labs()
```

```

    title = "Salary Distribution Across Departments",
    x = "Department",
    y = "Salary"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

# -----
# 5. Department-wise Violin Plot
# -----

ggplot(employee, aes(x = Department, y = Salary, fill = Department)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(
    title = "Salary Distribution by Department",
    x = "Department",
    y = "Salary"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
# -----
# 6. Summary Statistics
# -----

salary_summary <- employee %>%
  group_by(Department) %>%
  summarise(
    Mean = mean(Salary),
    Median = median(Salary),
    SD = sd(Salary),
    Minimum = min(Salary),
    Maximum = max(Salary),
    Range = max(Salary) - min(Salary)
  )

print(salary_summary)

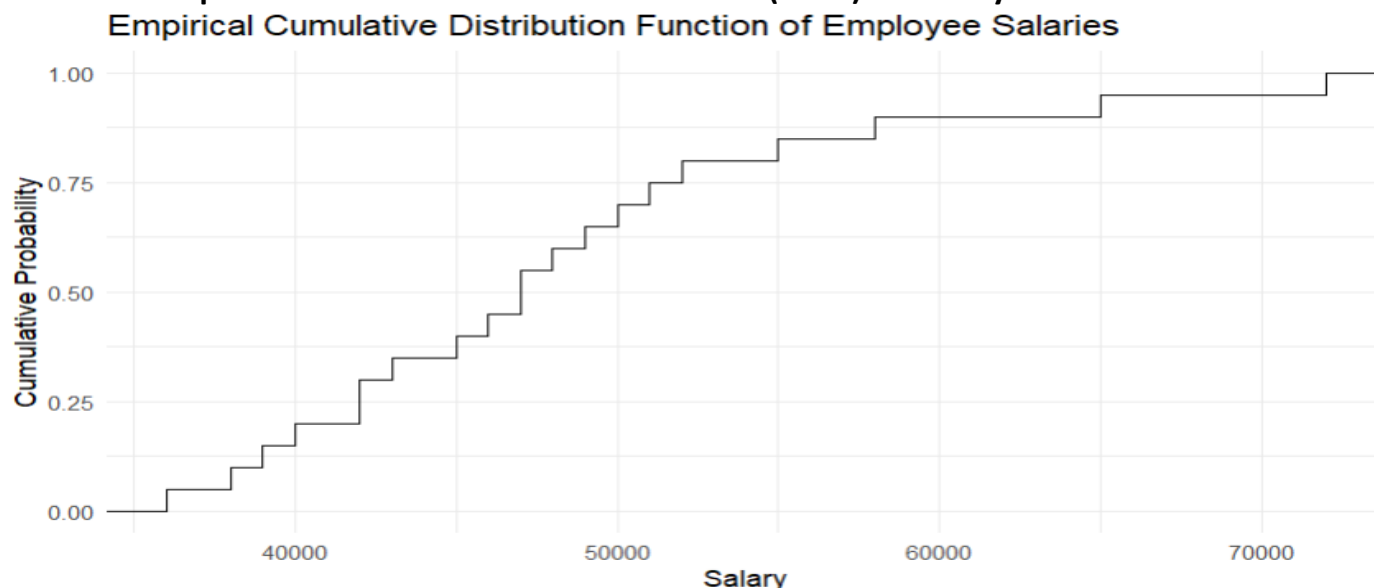
# Overall statistics
cat("Overall Mean Salary:", mean(employee$Salary), "\n")

```

```
cat("Overall Median Salary:", median(employee$Salary), "\n")
cat("Overall SD:", sd(employee$Salary), "\n")
```

Answers:

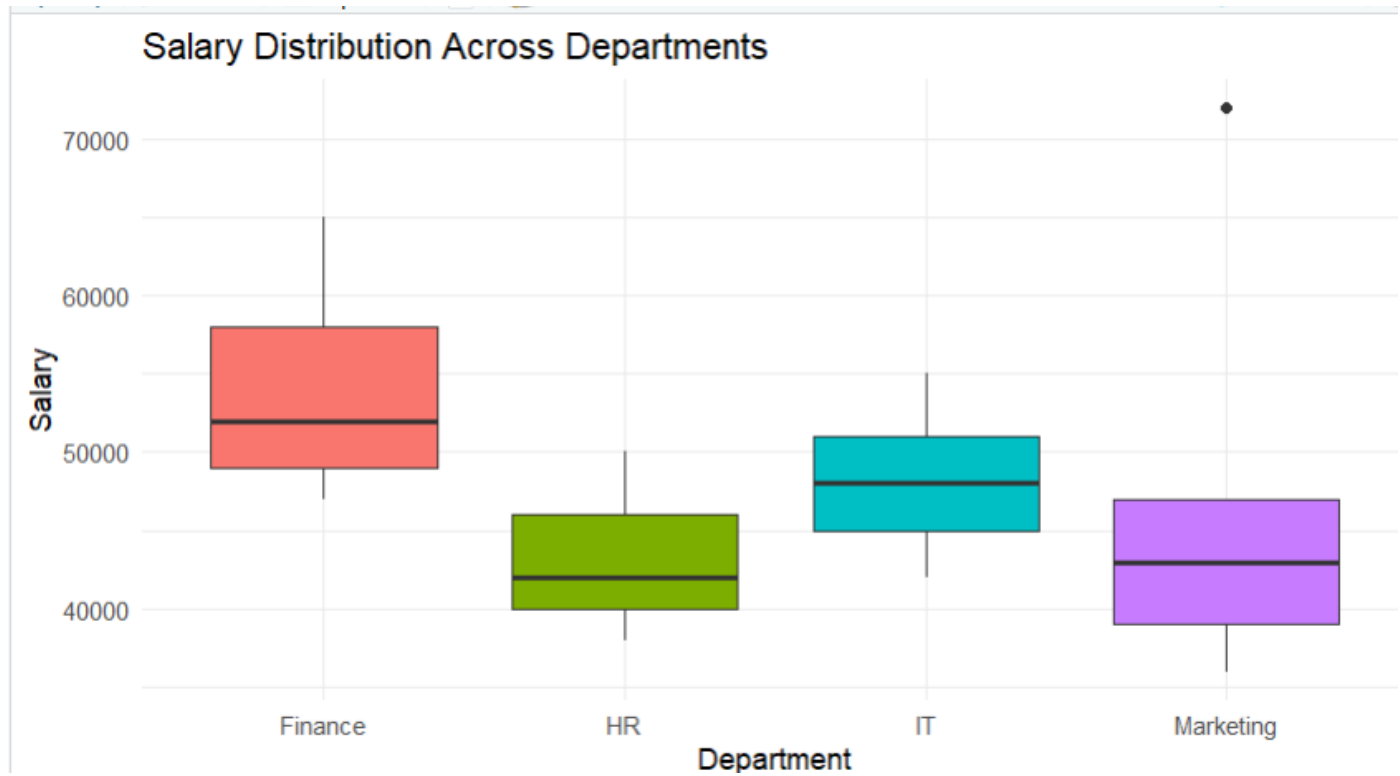
1. Create an empirical cumulative distribution function (ECDF) for Salary.



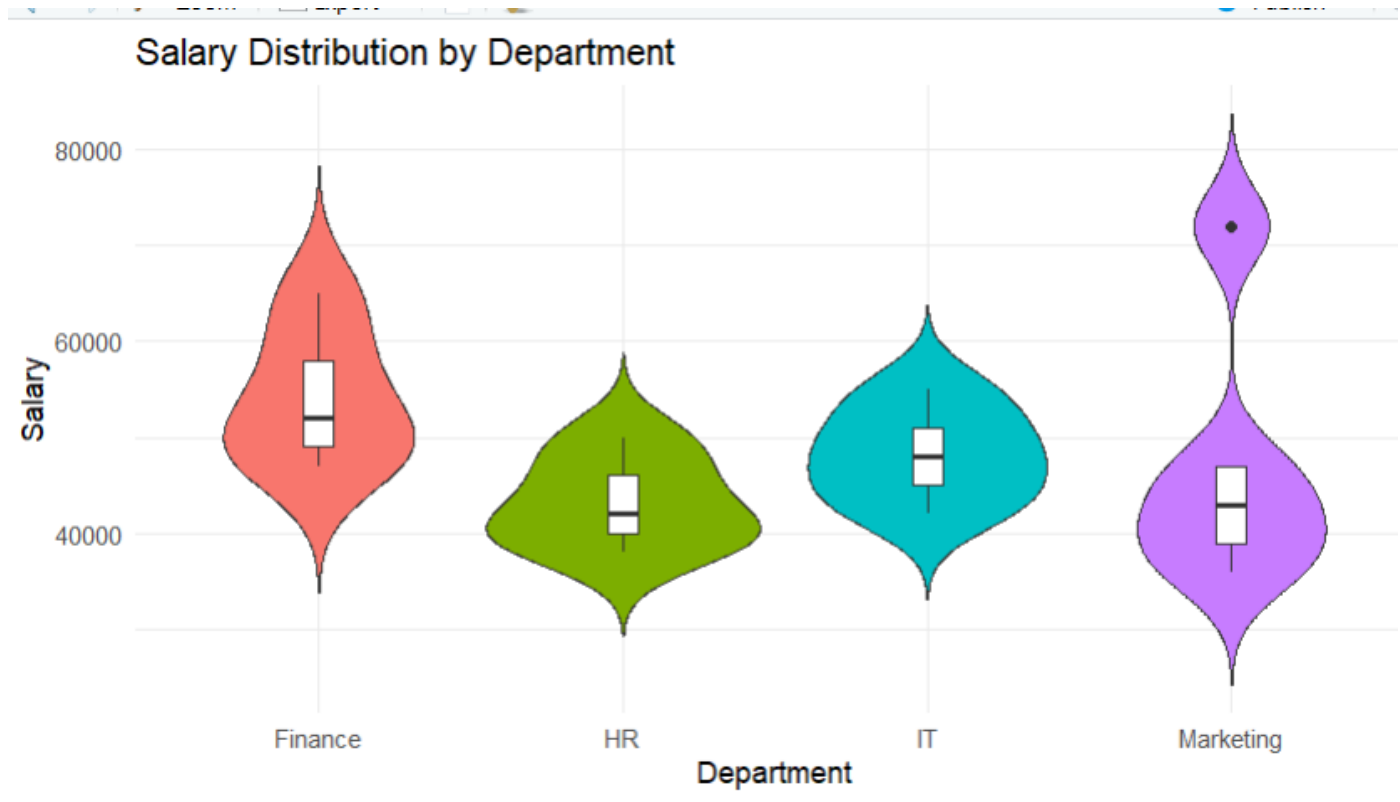
2. Create a Q-Q plot to assess whether Salary is approximately normally distributed.



3. Compare the salary distributions of the four departments using suitable distribution plots.



4. Identify any possible skewness or unusual observations and explain what the plots reveal.



Question 2 – Visualizing Many Distributions Using Bean Plots

Dataset: Student_Performance.csv

Assessment Task

Using the Student_Performance dataset, investigate how scores are distributed across courses.

1. Visualize the score distribution for all four courses at the same time.
2. Create bean plots (or an equivalent violin/density-based visualization if a bean-plot package is unavailable).
3. Create a second visualization with the distributions arranged along the vertical axis.
4. Compare the center, spread, and shape of the distributions and determine which course has the highest and most consistent performance.

Dataset:

Student_ID	Course	Study_Hours	Score
S001	Python	5	62
S002	Python	7	68
S003	Python	9	74
S004	Python	11	81
S005	Python	14	88
S006	Statistics	4	58
S007	Statistics	6	64
S008	Statistics	8	71
S009	Statistics	10	79
S010	Statistics	13	86
S011	DataViz	3	55
S012	DataViz	5	61
S013	DataViz	7	69
S014	DataViz	9	77
S015	DataViz	12	84
S016	MachineLearning	6	65
S017	MachineLearning	8	72
S018	MachineLearning	10	78

S019	MachineLearning	13	87
S020	MachineLearning	16	93

R Code:

```
# =====
# Q2: STUDENT PERFORMANCE DISTRIBUTION
# =====

library(ggplot2)
library(dplyr)
# -----
# 1. Create Dataset
# -----

student <- data.frame(
  Student_ID = paste0("S", sprintf("%03d", 1:20)),
  Course = c(
    rep("Python",5),
    rep("Statistics",5),
    rep("DataViz",5),
    rep("MachineLearning",5)
  ),
  Study_Hours = c(
    5,7,9,11,14,
    4,6,8,10,13,
    3,5,7,9,12,
    6,8,10,13,16
  ),
  Score = c(
    62,68,74,81,88,
    58,64,71,79,86,
    55,61,69,77,84,
    65,72,78,87,93
  )
)
```

```

print(student)
# -----
# 2. Horizontal Violin Plot
# -----
ggplot(student, aes(x = Course, y = Score, fill = Course)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = 0.12, fill = "white") +
  labs(
    title = "Score Distribution Across Courses",
    x = "Course",
    y = "Score"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
# -----
# 3. Vertical-axis Distribution
# -----
ggplot(student, aes(x = Score, y = Course, fill = Course)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(
    width = 0.12,
    orientation = "y",
    fill = "white"
  ) +
  labs(
    title = "Course Score Distributions Along Vertical Axis",
    x = "Score",
    y = "Course"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
# -----
# 4. Density Plot
# -----

```

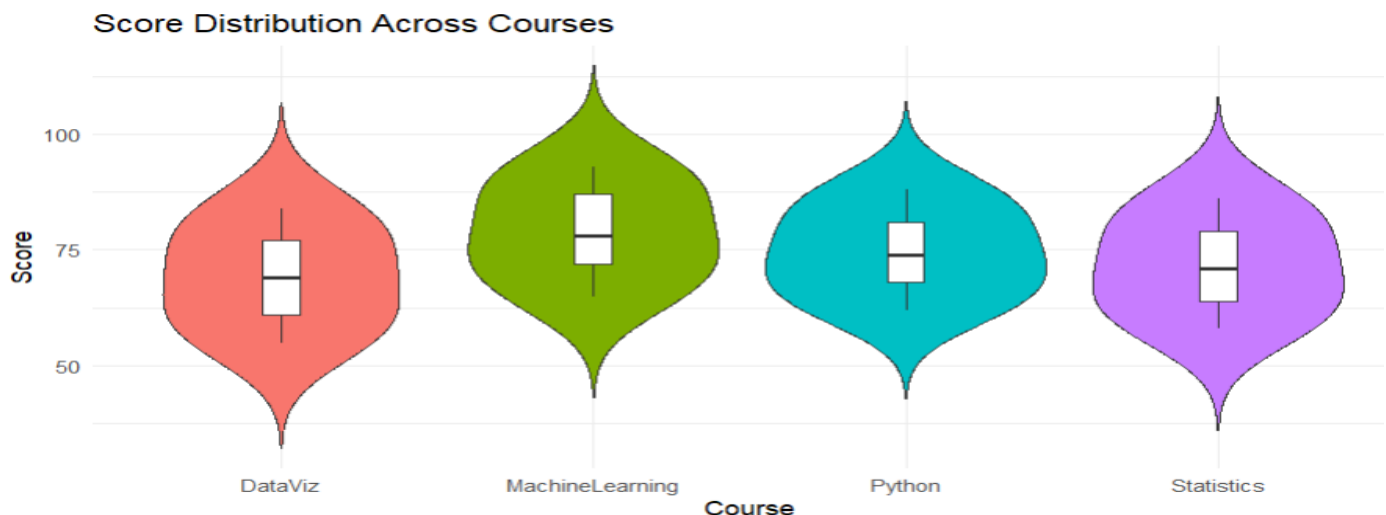
```

ggplot(student, aes(x = Score, fill = Course)) +
  geom_density(alpha = 0.4) +
  labs(
    title = "Density Distribution of Student Scores",
    x = "Score",
    y = "Density"
  ) +
  theme_minimal()
# -----
# 5. Summary Statistics
# -----
student_summary <- student %>%
  group_by(Course) %>%
  summarise(
    Mean = mean(Score),
    Median = median(Score),
    SD = sd(Score),
    Minimum = min(Score),
    Maximum = max(Score),
    Range = max(Score) - min(Score)
  )
print(student_summary)

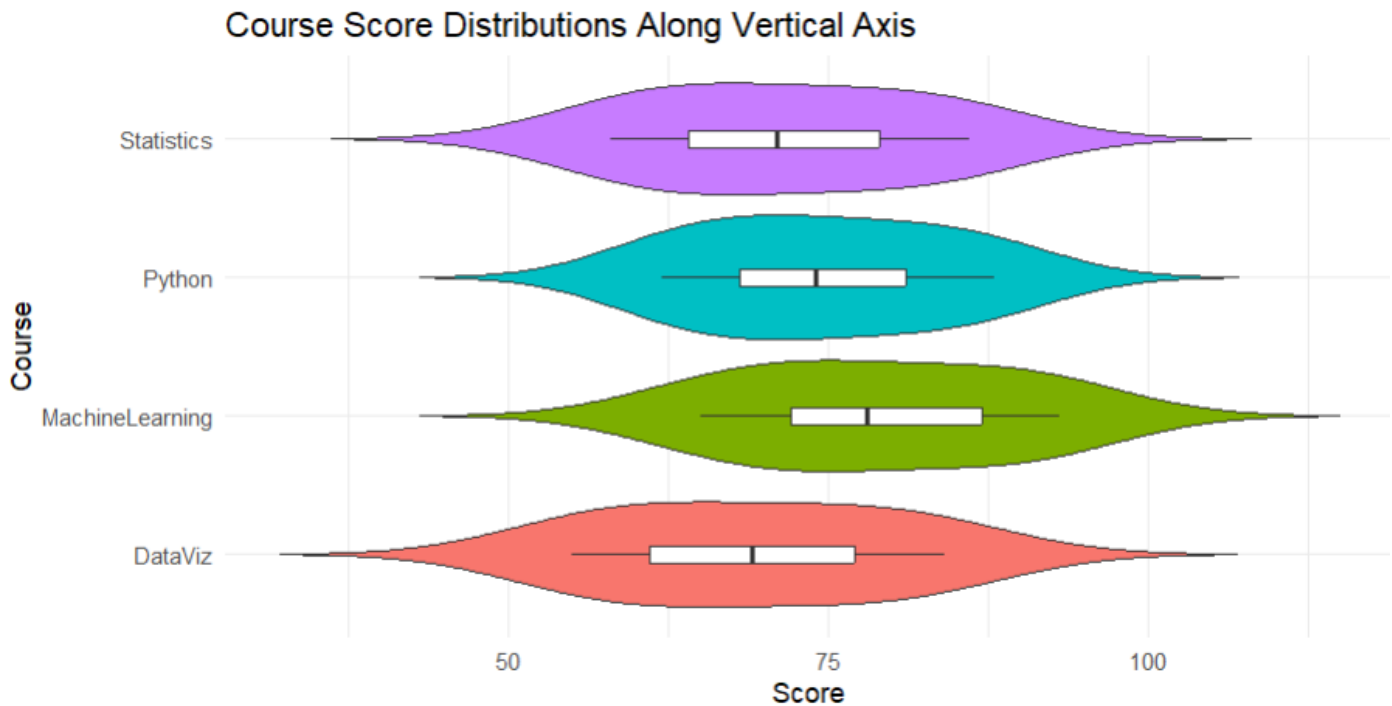
```

Answers:

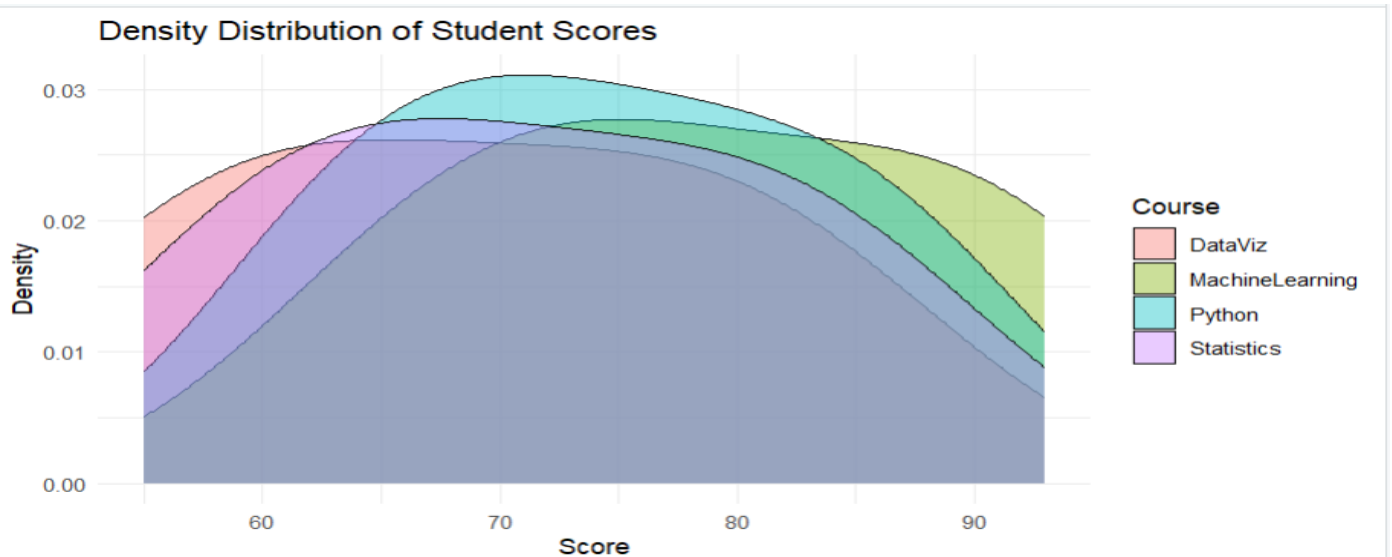
1. Visualize the score distribution for all four courses at the same time.



2. Create bean plots (or an equivalent violin/density-based visualization if a bean-plot package is unavailable).



3. Create a second visualization with the distributions arranged along the vertical axis.



4. Compare the center, spread, and shape of the distributions and determine which course has the highest and most consistent performance.

Course	Mean	Median	SD	Minimum	Maximum	Range
<i><chr></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>
DataViz	69.2	69	11.7	55	84	29
MachineLearning	79	78	11.2	65	93	28
Python	74.6	74	10.3	62	88	26
Statistics	71.6	71	11.2	58	86	28

Question 3 – Comparing Proportions with Pie, Side-by-Side and Stacked

Bars Dataset: Customer_Purchases.csv

Assessment Task

Using the Customer_Purchases dataset, analyze purchase proportions.

1. Create a pie chart showing the overall proportion of purchases by Product.
2. Create side-by-side bar charts comparing Product counts across Regions.
3. Create a stacked bar chart showing product composition within each Region.
4. Convert the stacked chart into a 100% stacked bar chart and compare proportions rather than raw counts.
5. Explain why one of these visualizations may be more appropriate than another for comparing regional proportions.

Dataset:

Customer_ID	Region	Product	Purchase_Type	Count
C001	North	Laptop	Online	25
C002	North	Phone	Online	35
C003	North	Tablet	Store	20
C004	South	Laptop	Store	30
C005	South	Phone	Online	40
C006	South	Tablet	Store	25
C007	East	Laptop	Online	28
C008	East	Phone	Store	32
C009	East	Tablet	Online	18
C010	West	Laptop	Store	22
C011	West	Phone	Online	38
C012	West	Tablet	Store	27

R Code:

```
#
```

```
=====
```

```
===
```

```
# Q3: CUSTOMER PURCHASE ANALYSIS
```

```
#
```

```
=====
```

```
===
```

```
library(ggplot2)
```

```
library(dplyr)
```

```
# -----
```

```
# 1. Create Dataset
```

```
# -----
```

```
customer <- data.frame(  
  Customer_ID = paste0("C", sprintf("%03d", 1:12)),
```

```
  Region = c(  
    "North","North","North",  
    "South","South","South",  
    "East","East","East",  
    "West","West","West"  
  ),
```

```
  Product = c(  
    "Laptop","Phone","Tablet",  
    "Laptop","Phone","Tablet",  
    "Laptop","Phone","Tablet",  
    "Laptop","Phone","Tablet"  
  ),
```

```
  Purchase_Type = c(  
    "Online","Online","Store",  
    "Store","Online","Store",  
    "Online","Store","Online",  
    "Store","Online","Store"  
  ),
```

```
Count = c(
  25,35,20,
  30,40,25,
  28,32,18,
  22,38,27
)
)
```

```
print(customer)
```

```
# -----
# 2. Overall Product Pie Chart
# -----
```

```
product_total <- customer %>%
  group_by(Product) %>%
  summarise(Total = sum(Count))
```

```
ggplot(product_total, aes(x = "", y = Total, fill = Product)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y") +
  labs(
    title = "Overall Purchase Proportion by Product",
    fill = "Product"
  ) +
  theme_void()
```

```
# -----
# 3. Side-by-Side Bar Chart
# -----
```

```
ggplot(customer,
  aes(x = Region, y = Count, fill = Product)) +
```

```
geom_col(position = "dodge") +  
labs(  
  title = "Product Counts Across Regions",  
  x = "Region",  
  y = "Purchase Count",  
  fill = "Product"  
) +  
theme_minimal()
```

```
# -----
```

```
# 4. Stacked Bar Chart
```

```
# -----
```

```
ggplot(customer,  
  aes(x = Region, y = Count, fill = Product)) +  
geom_col() +  
labs(  
  title = "Stacked Product Composition by Region",  
  x = "Region",  
  y = "Purchase Count",  
  fill = "Product"  
) +  
theme_minimal()
```

```
# -----
```

```
# 5. 100% Stacked Bar Chart
```

```
# -----
```

```
ggplot(customer,  
  aes(x = Region, y = Count, fill = Product)) +  
geom_col(position = "fill") +  
scale_y_continuous(labels = scales::percent) +  
labs(  
  title = "100% Stacked Bar Chart of Regional Product Proportions",
```



```

x = "Region",
y = "Percentage",
fill = "Product"
) +
theme_minimal()

# -----
# 6. Regional Proportions
# -----

regional <- customer %>%
  group_by(Region, Product) %>%
  summarise(Count = sum(Count), .groups = "drop") %>%
  group_by(Region) %>%
  mutate(Proportion = Count / sum(Count))

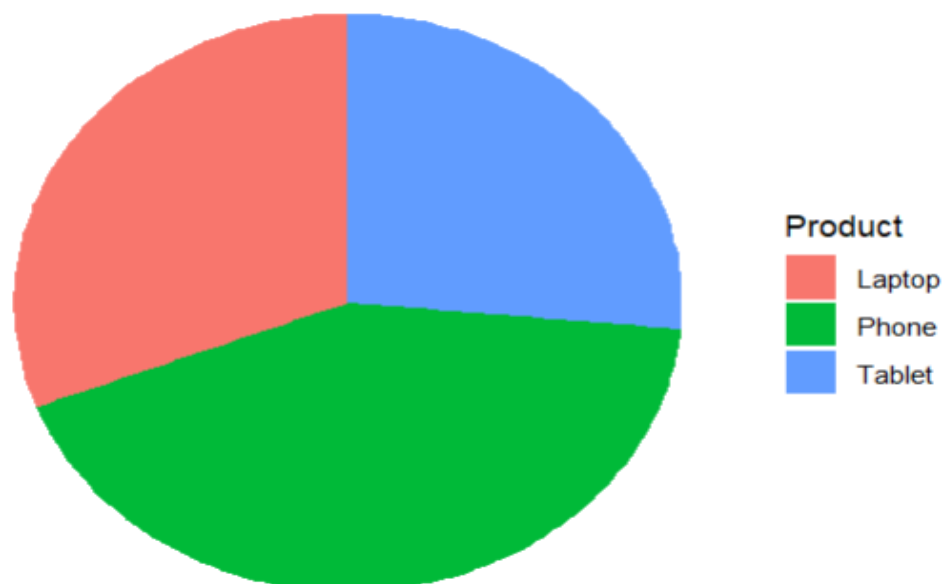
print(regional)

```

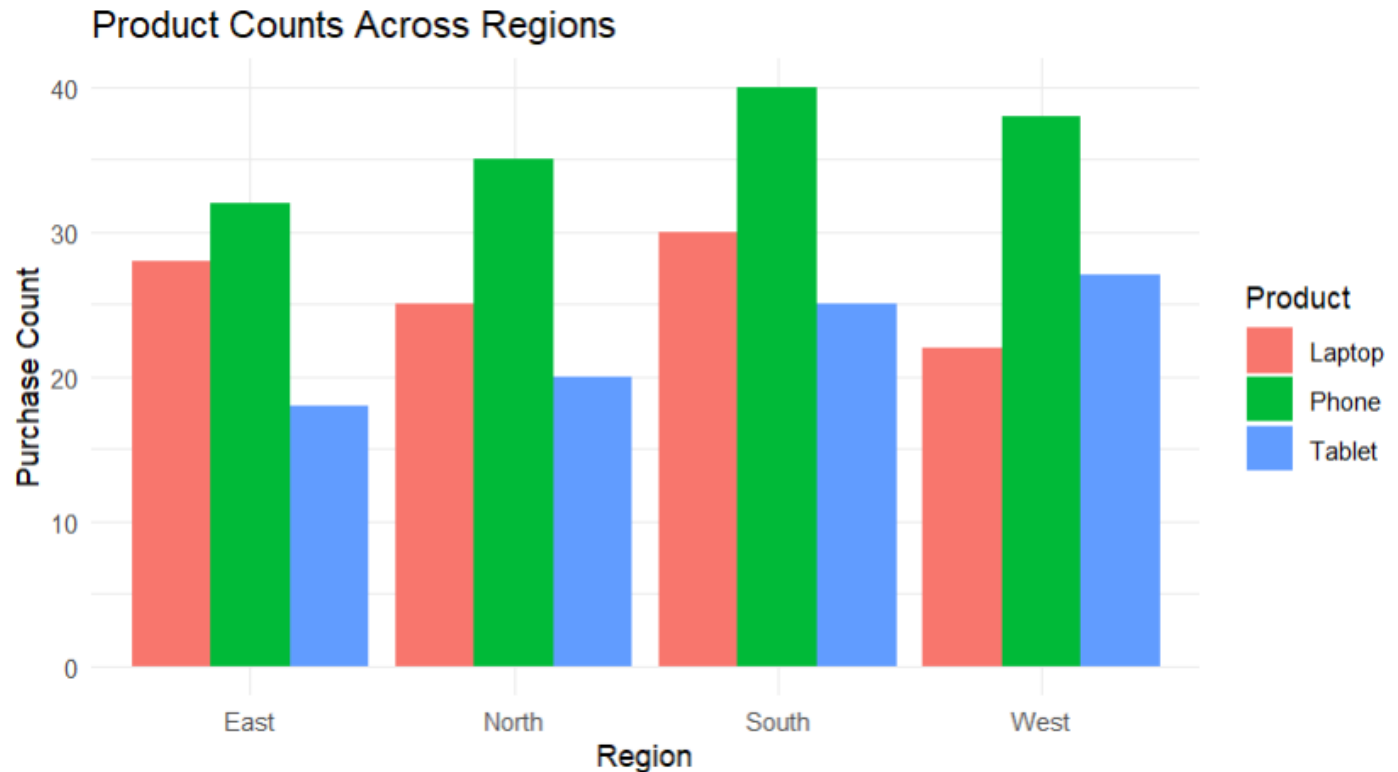
Answers:

1. Create a pie chart showing the overall proportion of purchases by Product.

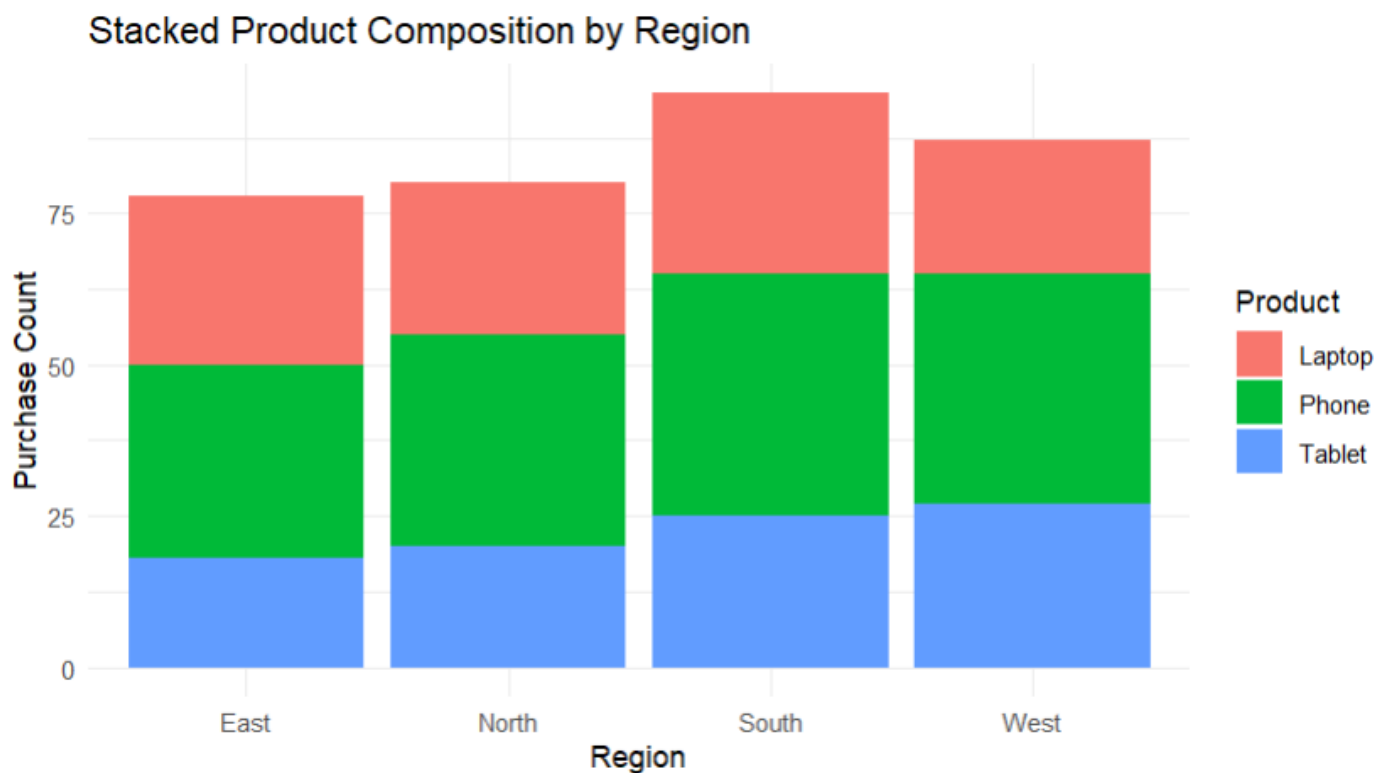
Overall Purchase Proportion by Product



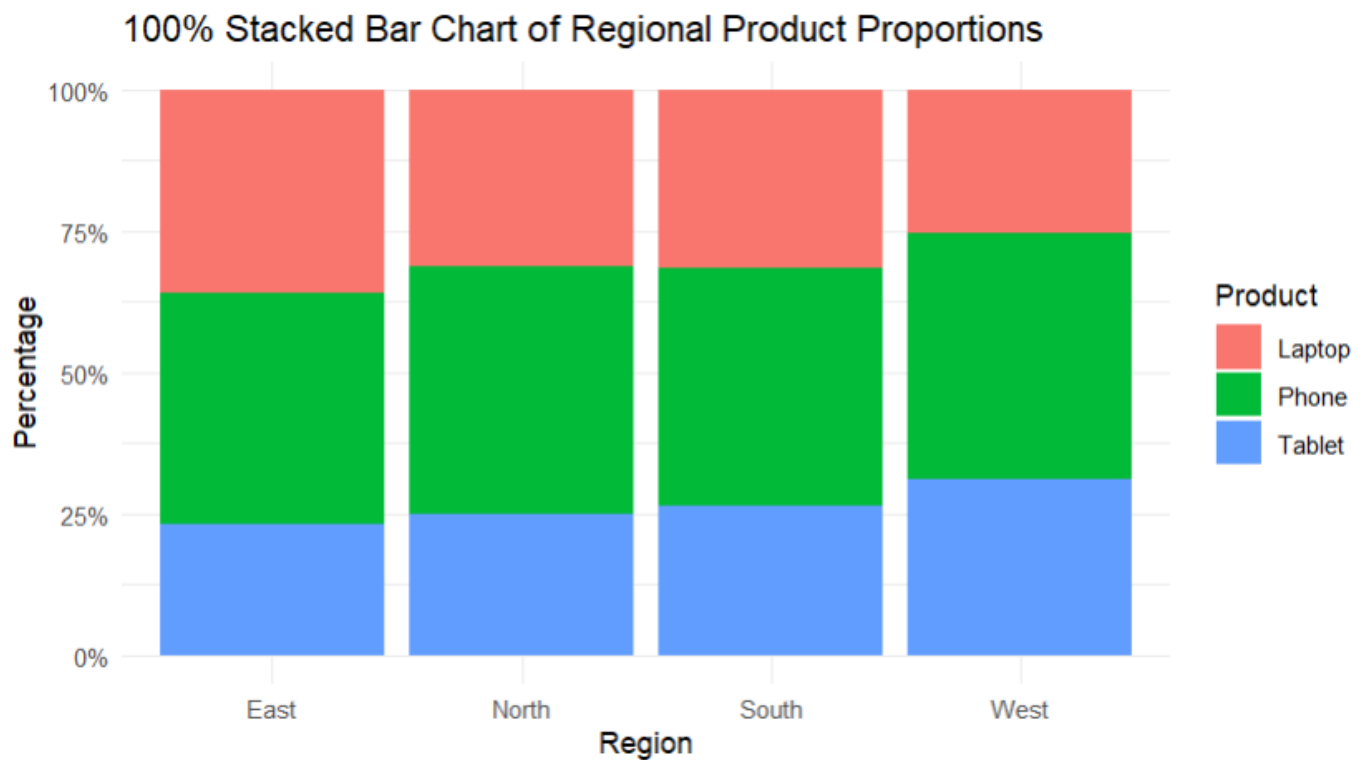
2. Create side-by-side bar charts comparing Product counts across Regions.



3. Create a stacked bar chart showing product composition within each Region.



4. Convert the stacked chart into a 100% stacked bar chart and compare proportions rather than raw counts.



5. Explain why one of these visualizations may be more appropriate than another for comparing regional proportions.

```
# Groups:   Region [4]
  Region Product Count
  <chr>   <chr>   <dbl>
1 East   Laptop    28
2 East   Phone     32
3 East   Tablet    18
4 North  Laptop    25
5 North  Phone     35
6 North  Tablet    20
7 South  Laptop    30
8 South  Phone     40
9 South  Tablet    25
10 West  Laptop    22
11 West  Phone     38
12 West  Tablet    27
```

Question 4 – Nested Proportions Using Treemap and Sunburst

Charts Dataset: Company_Sales.csv

Assessment Task

Using the Company_Sales dataset, visualize nested proportions in the sales hierarchy.

1. Create a treemap showing Region → Category → Subcategory, with rectangle size representing Sales.
2. Create a sunburst chart representing the same hierarchy.
3. Identify the largest region, category, and subcategory by sales.
4. Compare the readability of the treemap and sunburst chart for identifying nested proportions.
5. State one potential limitation of each visualization.

Dataset:

Region	Category	Subcategory	Sales
North	Electronics	Laptop	52000
North	Electronics	Phone	38000
North	Furniture	Chair	22000
North	Furniture	Desk	18000
South	Electronics	Laptop	46000
South	Electronics	Phone	42000
South	Furniture	Chair	25000
South	Furniture	Desk	21000
East	Electronics	Laptop	58000
East	Electronics	Phone	35000
East	Furniture	Chair	19000
East	Furniture	Desk	23000
West	Electronics	Laptop	49000
West	Electronics	Phone	44000
West	Furniture	Chair	27000
West	Furniture	Desk	20000

R Code:

```
# =====  
# Q4: COMPANY SALES HIERARCHY  
# =====
```

```
library(ggplot2)  
library(dplyr)  
library(treemapify)  
library(plotly)
```

```
# -----  
# 1. Create Dataset  
# -----
```

```
sales <- data.frame(  
  Region = c(  
    rep("North",4),  
    rep("South",4),  
    rep("East",4),  
    rep("West",4)  
  ),  
  
  Category = rep(  
    c(  
      "Electronics","Electronics",  
      "Furniture","Furniture"  
    ), 4  
  ),  
  
  Subcategory = rep(  
    c("Laptop","Phone","Chair","Desk"), 4  
  ),  
  
  Sales = c(  
    rep(1000,4),  
    rep(1500,4),  
    rep(2000,4),  
    rep(2500,4)
```

```

52000,38000,22000,18000,
46000,42000,25000,21000,
58000,35000,19000,23000,
49000,44000,27000,20000
)
)

print(sales)

# -----
# 2. Treemap
# -----

ggplot(
  sales,
  aes(
    area = Sales,
    fill = Category,
    label = paste(Subcategory, "\n", Sales)
  )
) +
  geom_treemap() +
  geom_treemap_text(
    colour = "white",
    place = "centre",
    grow = TRUE
  ) +
  labs(
    title = "Company Sales Treemap",
    fill = "Category"
  ) +
  theme_minimal()

# -----

```

3. Treemap by Region

```
ggplot(  
  sales,  
  aes(  
    area = Sales,  
    fill = Category,  
    subgroup = Region,  
    label = Subcategory  
  )  
) +  
  geom_treemap() +  
  geom_treemap_subgroup_border() +  
  geom_treemap_text(colour = "white") +  
  labs(  
    title = "Region → Category → Subcategory Sales Treemap"  
  ) +  
  theme_minimal()
```

4. Sunburst Chart

```
sunburst <- plot_ly(  
  sales,  
  type = "sunburst",  
  labels = ~Subcategory,  
  parents = ~Category,  
  values = ~Sales,  
  branchvalues = "total"  
)
```

```
sunburst <- sunburst %>%
```

```
layout(  
  title = "Company Sales Sunburst Chart"  
)
```

sunburst

```
# -----  
# 5. Summary  
# -----
```

```
region_summary <- sales %>%  
  group_by(Region) %>%  
  summarise(Total_Sales = sum(Sales)) %>%  
  arrange(desc(Total_Sales))
```

```
category_summary <- sales %>%  
  group_by(Category) %>%  
  summarise(Total_Sales = sum(Sales)) %>%  
  arrange(desc(Total_Sales))
```

```
subcategory_summary <- sales %>%  
  group_by(Subcategory) %>%  
  summarise(Total_Sales = sum(Sales)) %>%  
  arrange(desc(Total_Sales))
```

```
print(region_summary)  
print(category_summary)  
print(subcategory_summary)
```

Answers:

1. Create a treemap showing Region → Category → Subcategory, with rectangle size representing Sales.



2. Create a sunburst chart representing the same hierarchy.



3. Identify the largest region, category, and subcategory by sales.

```
# A tibble: 4 x 2
  Region Total_Sales
  <chr>      <dbl>
1 West      140000
2 East      135000
3 South     134000
4 North     130000
> |
```

4. Compare the readability of the treemap and sunburst chart for identifying nested proportions.

```

# A tibble: 2 x 2
  Category    Total_Sales
  <chr>      <dbl>
1 Electronics 364000
2 Furniture  175000
> |

```

5. State one potential limitation of each visualization.

- A major limitation of both [treemap and sunburst](#) visualizations is difficulty in making precise visual comparisons of sizes or proportions.
- Because human eyes struggle to accurately judge relative differences in irregular areas (rectangles in treemaps) or arc lengths/angles (segments in sunbursts) compared to a linear baseline, it is hard to tell which data categories are truly larger or by how much.

Question 5 – Flow-Based Proportions with Sankey Diagram and Parallel

Sets Dataset: Student_Career_Path.csv

Assessment Task

Using the Student_Career_Path dataset, analyze how students flow from specialization to internship status and placement sector.

1. Create a Sankey diagram showing Specialization → Internship → Placement_Sector, with flow width based on Students.
2. Create a parallel sets visualization for the same categorical relationships.
3. Identify the major flows and compare the placement patterns of students with and without internships.
4. Explain which visualization communicates the flow structure more clearly and why.
5. Discuss one way the chart could become misleading if proportions are represented using inappropriate scales, colors, or labels.

Dataset:

Student_Group	Specialization	Internship	Placement_Sector	Students
G1	Data Analytics	Yes	IT	18
G2	Data Analytics	Yes	Finance	12
G3	Data Analytics	No	IT	8
G4	AI	Yes	IT	20
G5	AI	Yes	Healthcare	10
G6	AI	No	IT	6

G7	Cyber Security	Yes	IT	16
G8	Cyber Security	Yes	Finance	9
G9	Cyber Security	No	Government	7
G10	Cloud Computing	Yes	IT	15
G11	Cloud Computing	Yes	Finance	8
G12	Cloud Computing	No	Government	5

R Code:

```
# =====
# Q5 - SANKEY + PARALLEL SETS
# =====
```

```
library(networkD3)
library(ggplot2)
library(dplyr)
library(ggalluvial)
```

```
# =====
# 1. DATASET
# =====
```

```
career <- data.frame(
  Student_Group = c(
    "G1","G2","G3","G4","G5","G6",
    "G7","G8","G9","G10","G11","G12"
  ),
```

```
  Specialization = c(
    "Data Analytics","Data Analytics","Data Analytics",
    "AI","AI","AI",
    "Cyber Security","Cyber Security","Cyber Security",
    "Cloud Computing","Cloud Computing","Cloud Computing"
  ),
```

```
  Internship = c(
    "Yes","Yes","No",
```

```

    "Yes","Yes","No",
    "Yes","Yes","No",
    "Yes","Yes","No"
  ),

  Placement_Sector = c(
    "IT","Finance","IT",
    "IT","Healthcare","IT",
    "IT","Finance","Government",
    "IT","Finance","Government"
  ),

  Students = c(
    18,12,8,
    20,10,6,
    16,9,7,
    15,8,5
  )
)

print(career)

# =====
# 2. SANKEY DIAGRAM
# =====

nodes <- data.frame(
  name = unique(c(
    career$Specialization,
    career$Internship,
    career$Placement_Sector
  ))
)

flow1 <- career %>%
  group_by(Specialization, Internship) %>%
  summarise(
    Students = sum(Students),

```

```

    .groups = "drop"
  ) %>%
  rename(
    source = Specialization,
    target = Internship
  )

flow2 <- career %>%
  group_by(Internship, Placement_Sector) %>%
  summarise(
    Students = sum(Students),
    .groups = "drop"
  ) %>%
  rename(
    source = Internship,
    target = Placement_Sector
  )

links_data <- bind_rows(flow1, flow2)

links <- links_data %>%
  mutate(
    source = match(source, nodes$name) - 1,
    target = match(target, nodes$name) - 1,
    value = Students
  )

sankey <- sankeyNetwork(
  Links = as.data.frame(links),
  Nodes = as.data.frame(nodes),
  Source = "source",
  Target = "target",
  Value = "value",
  NodeID = "name",
  fontSize = 13,
  nodeWidth = 30,
  sinksRight = TRUE
)

```

```
# Show Sankey
sankey
```

```
# =====
# 3. PARALLEL SETS
# =====
```

```
parallel_plot <- ggplot(
  career,
  aes(
    axis1 = Specialization,
    axis2 = Internship,
    axis3 = Placement_Sector,
    y = Students
  )
) +
  geom_alluvium(
    aes(fill = Internship),
    width = 1/12,
    alpha = 0.7
  ) +
  geom_stratum(
    width = 1/12,
    fill = "grey80",
    colour = "black"
  ) +
  geom_text(
    stat = "stratum",
    aes(label = after_stat(stratum)),
    size = 3
  ) +
  scale_x_discrete(
    limits = c(
      "Specialization",
      "Internship",
      "Placement Sector"
    ),
    expand = c(0.05, 0.05)
```

```

) +
labs(
  title = "Parallel Sets - Student Career Path",
  subtitle = "Specialization → Internship → Placement Sector",
  x = NULL,
  y = "Number of Students",
  fill = "Internship"
) +
theme_minimal()

```

```

# IMPORTANT: Display second graph
print(parallel_plot)

```

```

# =====
# 4. RESULTS
# =====

```

```

cat(
  "\nStudents with Internship:",
  sum(career$Students[career$Internship == "Yes"])
)

```

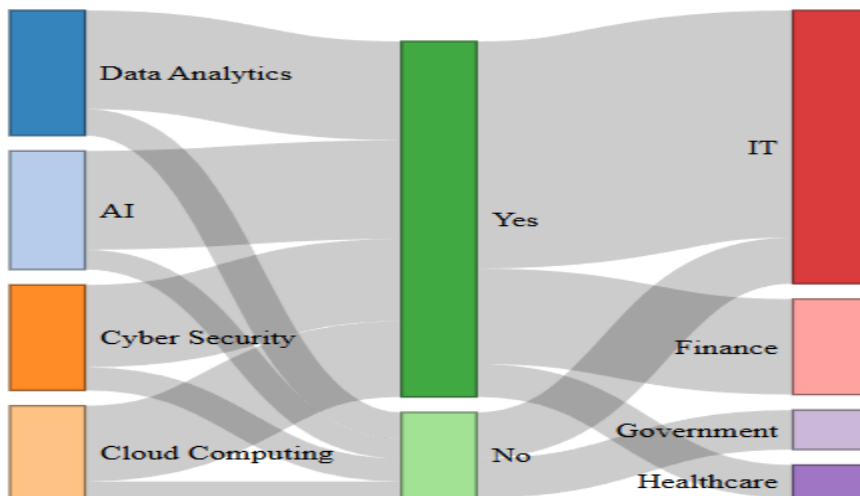
```

cat(
  "\nStudents without Internship:",
  sum(career$Students[career$Internship == "No"])
)
cat("\n")

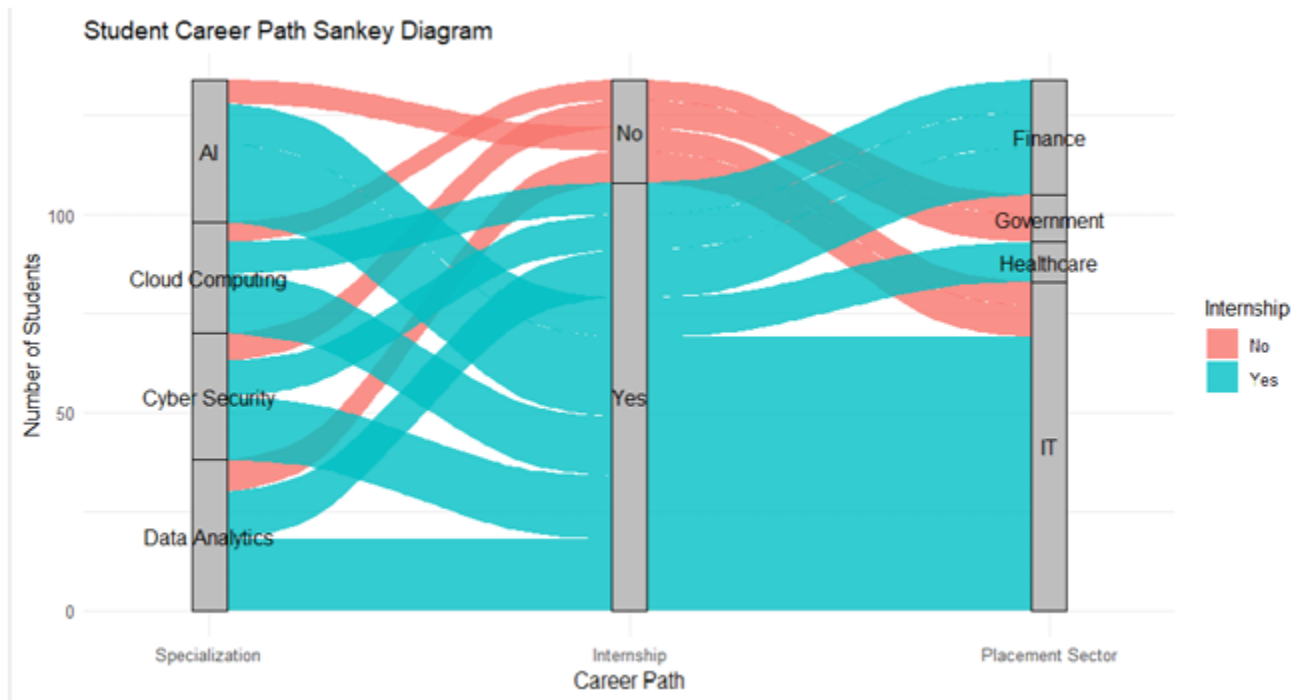
```

Answers:

1. Create a Sankey diagram showing Specialization → Internship → Placement_Sector, with flow width based on Students.



2. Create a parallel sets visualization for the same categorical relationships.



3. Identify the major flows and compare the placement patterns of students with and without internships.


```
> print(major_flows)
  Specialization Internship Placement_Sector Students
4             AI         Yes                IT        20
1 Data Analytics         Yes                IT        18
7 Cyber Security         Yes                IT        16
10 Cloud Computing         Yes                IT        15
2 Data Analytics         Yes                Finance     12
5             AI         Yes            Healthcare     10
8 Cyber Security         Yes                Finance      9
3 Data Analytics         No                 IT          8
11 Cloud Computing         Yes                Finance      8
9 Cyber Security         No            Government      7
6             AI         No                 IT          6
12 Cloud Computing         No            Government      5
> |
. . . . .
```

4. Explain which visualization communicates the flow structure more clearly and why.

Sankey diagram

The Sankey diagram communicates the flow structure more clearly.

It makes it easy to follow: Specialization → Internship → Placement Sector The width of each connection immediately shows how many students follow that path.

Parallel Sets

The parallel-sets chart is also useful, especially for comparing multiple categorical dimensions simultaneously, but the crossing lines can become harder to follow when there are many categories.

Therefore: Sankey diagram is better for understanding the overall flow of students through the career pathway.

5. Discuss one way the chart could become misleading if proportions are represented using inappropriate scales, colors, or labels.

- The chart can become misleading if the flow widths, colors, or labels do not accurately represent the proportions.
- For example, if a flow representing 5 students is displayed almost as wide as a flow representing 20 students, the viewer may incorrectly assume that both flows have similar importance.
- Similarly, using confusing colors or misleading percentage labels can make the comparison between internship and non-internship students difficult.

Expected Student Deliverables

- R script (.R) containing data creation/import, data preparation, analysis, and visualization code.

All required plots with meaningful titles, axis labels, legends, and appropriate scales. • A short

interpretation for each visualization.

- A brief comparison of alternative visualization methods where requested.
- A concluding statement identifying important patterns, differences, or potential misleading visual encodings.

Suggested Assessment Criteria

Criterion	Weight
Dataset preparation	20%
Correct R programming and use of visualization packages	25%
Appropriate choice and construction of visualizations	25%
Interpretation and analytical reasoning	20%
Clarity, labeling, and presentation	10%